

PUPPET - 90-MINUTE HANDS-ON LAB

Student Step-by-Step Instructions | macOS + Windows (WSL2)

Key rule: Windows students run Puppet inside Ubuntu/WSL2; macOS students run Puppet directly in Terminal.

The lab covers Puppet architecture, components, installation, manifests/coding style, classes & modules, module file serving, templates, custom functions, resource providers, idempotency, user creation and package installation.

1. Windows - verify WSL2 + Ubuntu

- Open PowerShell or Windows Terminal. Use this only to check WSL; Puppet commands are run inside Ubuntu later.

```
wsl --status
wsl --list --verbose
```

- Ubuntu should be listed with VERSION 2. If Ubuntu is missing:

```
wsl --install -d Ubuntu
```

- Restart Windows if requested. Launch Ubuntu from the Start menu and finish the first-time username/password setup.

2. macOS - check the machine

```
sw_vers
uname -m
which brew || brew --version
```

- Homebrew is needed only for the macOS package demo. If brew is missing, install Homebrew using its official installer and follow the printed shellenv instructions.

3. Install Puppet Core

Use the official Puppet Core installer/package supplied by your instructor for this lab. Current Puppet Core packages are distributed through protected repositories, so the classroom package is provided in advance.

3.1 macOS - install

```
sudo installer -pkg "$HOME/Downloads/puppet-agent-installer.pkg" -target /
export PATH="/opt/puppetlabs/bin:$PATH"
puppet --version
factor --version
puppet apply --version
```

- If your file is inside a .dmg, open the DMG first and use the puppet-agent-installer.pkg inside it.

3.2 Windows + WSL2 - install

```
cat /etc/os-release
uname -m
sudo apt update
sudo apt install -y ./Downloads/puppet-agent*.deb
source /etc/profile.d/puppet-agent.sh 2>/dev/null || true
export PATH="/opt/puppetlabs/bin:$PATH"
puppet --version
factor --version
puppet apply --version
```

- If the .deb is somewhere else, replace the path. Use the package matching your WSL architecture.

4. Get and enter the supplied lab files

Extract Puppet_Fundamentals_Hands_On_Files_Simple.zip. The extracted folder contains a common/ directory with all manifests and the module.

4.1 macOS

```
cd ~
unzip ~/Downloads/Puppet_Fundamentals_Hands_On_Files_Simple.zip
cd ~/puppet_lab_simple/common
pwd
ls
```

4.2 Windows + WSL2

```
cd ~
cp /mnt/c/Users/<WINDOWS_USERNAME>/Downloads/Puppet_Fundamentals_Hands_On_Files_Simple.zip .
```

```
unzip Puppet_Fundamentals_Hands_On_Files_Simple.zip
cd ~/puppet_lab_simple/common
pwd
ls
```

5. Check Puppet before the practical

```
puppet --version
factor --version
puppet apply --version
puppet describe file | head -n 20
```

- If Puppet prints version information and a description of the file resource, continue.

6. Practical 1 - resource + manifest + coding style

- Open 01_basics.pp and notice: resource type, resource title, attributes, commas and braces.

```
puppet apply --modulepath=modules --noop 01_basics.pp
sudo puppet apply --modulepath=modules 01_basics.pp
cat /tmp/puppet-lab/hello.txt
ls -l /tmp/puppet-lab
```

- Run the same manifest again. The second run should show no new file change.

```
sudo puppet apply --modulepath=modules 01_basics.pp
```

- Teaching point: Puppet describes the desired state. This is why repeated runs can be idempotent.

7. Practical 2 - user creation (required activity)

```
sudo puppet apply --modulepath=modules 02_user.pp
puppet resource user devopsstudent
```

- If the user already exists, Puppet should report that no creation is needed.

8. Practical 3 - package installation (required activity)

- macOS: make sure Homebrew is available. Windows: stay inside Ubuntu/WSL2.

```
brew --version # macOS only
sudo puppet apply --modulepath=modules 03_package.pp
puppet resource package tree
```

- The manifest uses the Homebrew provider on macOS and apt on Ubuntu/WSL2. The same Puppet resource is implemented by an OS-specific provider.

9. Practical 4 - file resource

```
sudo puppet apply --modulepath=modules 04_file.pp
cat /tmp/puppet-lab/app.conf
```

10. Classes + modules

- The supplied puppet_lab module contains a class in modules/puppet_lab/manifests/init.pp.

```
sudo puppet apply --modulepath=modules 05_class.pp
cat /tmp/puppet-lab/module-message.txt
find modules/puppet_lab -maxdepth 2 -type f | sort
```

- Teaching point: a module packages reusable Puppet code, classes, files, templates and functions.

11. File server concept + puppet:///

```
sudo puppet apply --modulepath=modules 06_file_server.pp
cat /tmp/puppet-lab/hello-from-module.txt
```

- The file comes from the module files/ directory. The puppet:/// URL points to the module file. This works with puppet apply, so no standalone Puppet Server is needed for this demo.

12. Templates

```
sudo puppet apply --modulepath=modules 07_template.pp
cat /tmp/puppet-lab/welcome.html
```

- The EPP template receives student_name and the node hostname, then generates the file content.

13. Custom function

```
sudo puppet apply --modulepath=modules 08_function.pp
```

- Open modules/puppet_lab/functions/greeting.pp. A Puppet-language function takes an argument and returns a value.

14. Type + provider

```
sudo puppet apply --modulepath=modules 09_provider.pp
puppet resource package tree
```

- Teaching point: the resource type describes the desired state; the provider implements the operations for the operating system.

15. Final challenge - put it together

```
sudo puppet apply --modulepath=modules 10_final_challenge.pp
cat /tmp/puppet-lab/final.txt
puppet resource user devopsstudent
puppet resource package tree
```

- Run it a second time and observe that already-correct resources do not need to change.

16. Fast troubleshooting

- puppet: command not found -> export PATH="/opt/puppetlabs/bin:\$PATH" and retry. On WSL Linux, also try: source /etc/profile.d/puppet-agent.sh
- Permission denied -> rerun system-changing manifests with sudo.
- brew: command not found on macOS -> install Homebrew and reopen Terminal.
- tree package fails on WSL -> run sudo apt update, then rerun the Puppet package manifest.
- User already exists -> this is normally fine; verify with puppet resource user devopsstudent.
- puppet:/// file not found -> make sure you are inside ~/puppet_lab_simple/common and use --modulepath=modules.
- Template/function not found -> make sure the modules/puppet_lab directory and its subfolders were extracted correctly.
- Windows command not working -> make sure you are in the Ubuntu/WSL terminal, not PowerShell.

17. Final checklist

- ☐ Puppet version
- ☐ Facter version
- ☐ puppet apply
- ☐ Basic resource
- ☐ Idempotency / repeat run
- ☐ User creation
- ☐ Package installation
- ☐ File resource
- ☐ Classes
- ☐ Modules
- ☐ puppet:/// file
- ☐ EPP template
- ☐ Custom function
- ☐ Resource + provider
- ☐ Final challenge

Official validation notes: This guide is aligned to current Puppet Core 9.0.0 documentation available in September 2026. Puppet Core supports macOS and Ubuntu; puppet apply runs locally without a separate Puppet Server. Module files can be served with puppet:/// under puppet apply, and Puppet-language functions belong in a module functions/ directory.

```
sudo puppet apply --modulepath=modules manifests/01_basics.pp
sudo puppet apply --modulepath=modules manifests/02_user.pp
puppet apply --modulepath=modules manifests/03_package.pp
sudo puppet apply --modulepath=modules manifests/04_file.pp
sudo puppet apply --modulepath=modules manifests/05_class.pp
sudo puppet apply --modulepath=modules manifests/06_file_server.pp
sudo puppet apply --modulepath=modules manifests/07_template.pp
sudo puppet apply --modulepath=modules manifests/08_function.pp
sudo puppet apply --modulepath=modules manifests/09_provider.pp
sudo puppet apply --modulepath=modules manifests/10_final_challenge.pp
```